MIPT & YSDA · AUTUMN 2026

Deep Generative Models

Lecture 8

Roman Isachenko

Moscow Institute of Physics and Technology
Yandex School of Data Analysis

Recap of Previous Lecture

Fix a schedule $0 < \beta_t < 1$. The noises $\boldsymbol{\epsilon}_t \sim \mathcal{N}(0, \mathbf{I})$ are independent across t and of $\mathbf{x}_0$.

$$\mathbf{x}_t = \sqrt{1-\beta_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\boldsymbol{\epsilon}_t, \quad \boldsymbol{\epsilon}_t \sim \mathcal{N}(0, \mathbf{I})$$

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\sqrt{1-\beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I})$$

Exact Conditional Distribution

Let $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s = \prod_{s=1}^{t}(1-\beta_s)$. Then

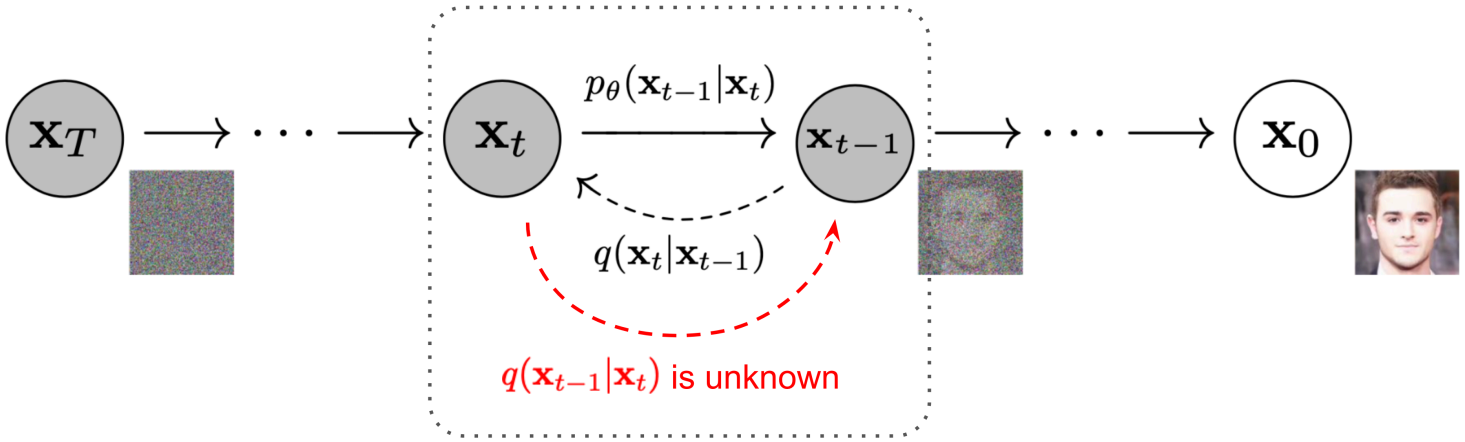
$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}\,\mathbf{x}_0, (1-\bar{\alpha}_t)\mathbf{I})$$

Connection to NCSN: $\mathbf{x}_t = \mathbf{x}_0 + \sigma_t\boldsymbol{\epsilon}$ adds Gaussian noise; DDPM also scales the signal by $\sqrt{\bar{\alpha}_t}$.

For finite T, choose $\bar{\alpha}_T \ll 1$ so the residual signal $\sqrt{\bar{\alpha}_T}\mathbf{x}_0$ is negligible; then $q(\mathbf{x}_T) \approx \mathcal{N}(0, \mathbf{I})$.

Sohl-Dickstein J. Deep Unsupervised Learning using Nonequilibrium Thermodynamics, 2015
Chan S. Tutorial on Diffusion Models for Imaging and Vision, 2024
Song Y. et al. Generative Modeling by Estimating Gradients of the Data Distribution, 2019

Recap of Previous Lecture



Reverse Process (Ancestral Sampling)

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t) = \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})q(\mathbf{x}_{t-1})}{q(\mathbf{x}_t)}$$
$$\approx p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}\left(\boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t), \boldsymbol{\sigma}^2_{\boldsymbol{\theta},t}(\mathbf{x}_t)\right)$$

If β_t is sufficiently small, $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$ is approximately Gaussian.

Recap of Previous Lecture

Forward Process

1. $\mathbf{x}_0 \sim p_{\text{data}}(\mathbf{x}_0)$
2. $\mathbf{x}_t = \sqrt{1-\beta_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\boldsymbol{\epsilon}_t$
3. $q(\mathbf{x}_T) \approx \mathcal{N}(0, \mathbf{I})$

Reverse Process

1. $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$
2. $\mathbf{x}_{t-1} = \boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \boldsymbol{\sigma}_{\boldsymbol{\theta},t}(\mathbf{x}_t)\boldsymbol{\epsilon}_t$
3. $\mathbf{x}_0 \sim p_{\boldsymbol{\theta}}(\mathbf{x}_0) \approx p_{\text{data}}(\mathbf{x}_0)$

SNR and Log-SNR

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I}).$$

$$\text{SNR}(t) = \frac{\bar{\alpha}_t}{1-\bar{\alpha}_t}.$$

$$\lambda_t = \log \text{SNR}(t), \qquad \bar{\alpha}_t = \frac{1}{1+e^{-\lambda_t}}.$$

$\lambda_t > 0$: signal dominates; $\lambda_t = 0$: equal scales; $\lambda_t < 0$: noise dominates.

Weng L. What are Diffusion Models?, blog post, 2021
Kingma D. et al. Variational Diffusion Models, 2021

Recap of Previous Lecture

Forward process: with a suitable noise schedule, $q(\mathbf{x}_T) \approx \mathcal{N}(0, \mathbf{I})$ as the residual signal becomes negligible:

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\sqrt{1-\beta_t}\,\mathbf{x}_{t-1}, \beta_t\,\mathbf{I});$$
$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}\,\mathbf{x}_0, (1-\bar{\alpha}_t)\,\mathbf{I}).$$

Reverse process refers to an intractable distribution that can be approximated by a normal distribution (with unknown parameters) for small β_t:

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t) = \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})q(\mathbf{x}_{t-1})}{q(\mathbf{x}_t)} \approx \mathcal{N}\left(\boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t), \boldsymbol{\sigma}^2_{\boldsymbol{\theta},t}(\mathbf{x}_t)\right)$$

Conditioned reverse process is a normal distribution with known parameters, describing how to denoise a noisy image $\mathbf{x}_t$ when we know the clean image $\mathbf{x}_0$.

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t\,\mathbf{I})$$

Recap of Previous Lecture

- $\mathbf{z} = (\mathbf{x}_1, \ldots, \mathbf{x}_T)$ represents the latent variables.
- **Fixed encoder:** the noise schedule determines the variational posterior:

$$q(\mathbf{z}|\mathbf{x}) = q(\mathbf{x}_1, \ldots, \mathbf{x}_T|\mathbf{x}_0) = \prod_{t=1}^{T} q(\mathbf{x}_t|\mathbf{x}_{t-1}).$$

- **One-step decoder and learned Markov-chain prior:**

$$p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z}) = p_{\boldsymbol{\theta}}(\mathbf{x}_0|\mathbf{x}_1); \quad p_{\boldsymbol{\theta}}(\mathbf{z}) = \prod_{t=2}^{T} p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t) \cdot p(\mathbf{x}_T)$$

The encoder has no trainable parameters; only θ is optimized.

Das A. An Introduction to Diffusion Probabilistic Models, blog post, 2021
Ho J. Denoising Diffusion Probabilistic Models, 2020

Recap of Previous Lecture

Diffusion ELBO (Fixed Encoder)

$$\log p_{\boldsymbol{\theta}}(\mathbf{x}) \geq \mathbb{E}_{q(\mathbf{z}|\mathbf{x})} \log \frac{p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z})}{q(\mathbf{z}|\mathbf{x})} = \mathcal{L}_{\boldsymbol{\phi},\boldsymbol{\theta}}(\mathbf{x}) \to \max_{\boldsymbol{\theta}}$$

$$\mathcal{L}_{\boldsymbol{\phi},\boldsymbol{\theta}}(\mathbf{x}) = \mathbb{E}_{q(\mathbf{x}_1|\mathbf{x}_0)} \log p_{\boldsymbol{\theta}}(\mathbf{x}_0|\mathbf{x}_1) - D_{\mathrm{KL}}\big(q(\mathbf{x}_T|\mathbf{x}_0)\|p(\mathbf{x}_T)\big) - \sum_{t=2}^{T} \underbrace{\mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} D_{\mathrm{KL}}\big(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)\|p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)\big)}_{\mathcal{L}_t}$$

Reconstruction: the decoder recovers $\mathbf{x}_0$ from $\mathbf{x}_1$.
For a continuous Gaussian decoder with fixed covariance $\sigma_1^2\mathbf{I}$, $\sigma_1^2 > 0$:

$$-\mathbb{E}_{q(\mathbf{x}_1|\mathbf{x}_0)} \log p_{\boldsymbol{\theta}}(\mathbf{x}_0|\mathbf{x}_1) = \frac{1}{2\sigma_1^2} \mathbb{E}_{q(\mathbf{x}_1|\mathbf{x}_0)} \big\|\mathbf{x}_0 - \boldsymbol{\mu}_{\boldsymbol{\theta},1}(\mathbf{x}_1)\big\|^2 + \text{const}.$$

Prior matching: constant in $\boldsymbol{\theta}$, since q and $p(\mathbf{x}_T) = \mathcal{N}(0, \mathbf{I})$ are fixed.

Intermediate denoising: match each learned reverse step to its known conditioned target, $t \geq 2$.

Das A. An Introduction to Diffusion Probabilistic Models, blog post, 2021
Ho J. Denoising Diffusion Probabilistic Models, 2020

Recap of Previous Lecture

Gaussian Reverse Steps

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}\big(\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t \mathbf{I}\big), \qquad p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}\big(\boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t), \tilde{\beta}_t \mathbf{I}\big).$$

For $t \geq 2$, fixing the model covariance to the target covariance gives

$$\mathcal{L}_t = \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} \left[\frac{1}{2\tilde{\beta}_t} \left\| \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) - \boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t) \right\|^2 \right].$$

Known Regression Target

$$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}(1-\alpha_t)}{1-\bar{\alpha}_t}\mathbf{x}_0,$$

$$\tilde{\beta}_t = \frac{(1-\alpha_t)(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}.$$

Learn the reverse mean by weighted regression onto the conditioned forward-process mean.

Recap of Previous Lecture

$$-\mathcal{L}_{\boldsymbol{\phi},\boldsymbol{\theta}}(\mathbf{x}_0) = -\mathbb{E}_{q(\mathbf{x}_1|\mathbf{x}_0)} \log p_{\boldsymbol{\theta}}(\mathbf{x}_0|\mathbf{x}_1) + D_{\mathrm{KL}}\big(q(\mathbf{x}_T|\mathbf{x}_0)\|p(\mathbf{x}_T)\big) + \sum_{t=2}^{T} \mathcal{L}_t.$$

Training

1. Sample $\mathbf{x}_0 \sim p_{\text{data}}$, $t \sim \text{Uniform}\{2, \dots, T\}$.
2. Sample $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$; construct $\mathbf{x}_1$, $\mathbf{x}_t$.
3. Minimize $\hat{\mathcal{L}}$ by SGD: the omitted prior KL is independent of $\boldsymbol{\theta}$.

$$\mathbf{x}_s = \sqrt{\bar{\alpha}_s}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_s}\boldsymbol{\epsilon}, \quad s \in \{1, t\}.$$

$$\hat{\mathcal{L}} = -\log p_{\boldsymbol{\theta}}(\mathbf{x}_0|\mathbf{x}_1) + \frac{T-1}{2\tilde{\beta}_t}\left\|\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) - \boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t)\right\|^2.$$

$T-1$: uniform timestep average → sum.

Sampling

1. Sample $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$.
2. Denoise $\mathbf{x}_{t-1} = \boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \sqrt{\tilde{\beta}_t}\boldsymbol{\epsilon}_t$.
3. Return the mean $\hat{\mathbf{x}}_0 = \boldsymbol{\mu}_{\boldsymbol{\theta},1}(\mathbf{x}_1)$.

For $t = T, \dots, 2$, sample the **reverse kernel**:

$$p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}\big(\boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t), \tilde{\beta}_t\mathbf{I}\big).$$

Use fresh $\boldsymbol{\epsilon}_t \sim \mathcal{N}(0, \mathbf{I})$ at each step.

Outline

01 Diffusion Parameterizations

02 Denoising Diffusion Probabilistic Model (DDPM)

03 From DDPM to DDIM

04 Model Guidance

Classifier Guidance
Classifier-Free Guidance

Outline

01 **Diffusion Parameterizations**

02 Denoising Diffusion Probabilistic Model (DDPM)

03 From DDPM to DDIM

04 Model Guidance

Classifier Guidance
Classifier-Free Guidance

Reparametrization of DDPM

$$\mathcal{L}_t = \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)}\left[\frac{1}{2\tilde{\beta}_t}\left\|\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) - \boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t)\right\|^2\right]$$

$$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\cdot\mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}(1-\alpha_t)}{1-\bar{\alpha}_t}\cdot\mathbf{x}_0$$

Forward Noising

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\cdot\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\cdot\boldsymbol{\epsilon}$$

Recover Clean Data

$$\mathbf{x}_0 = \frac{\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\cdot\boldsymbol{\epsilon}}{\sqrt{\bar{\alpha}_t}}$$

Why reparametrize? Predict the Gaussian noise we add ourselves during training; compute the reverse mean analytically from this prediction.

$$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \boldsymbol{\epsilon}) = \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\cdot\mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}(1-\alpha_t)}{1-\bar{\alpha}_t}\cdot\left(\frac{\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\cdot\boldsymbol{\epsilon}}{\sqrt{\bar{\alpha}_t}}\right)$$

$$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \boldsymbol{\epsilon}) = \frac{1}{\sqrt{\alpha_t}}\cdot\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{\alpha_t(1-\bar{\alpha}_t)}}\cdot\boldsymbol{\epsilon}$$

Ho J. Denoising Diffusion Probabilistic Models, 2020

Reparametrization of DDPM

$$\mathcal{L}_t = \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)}\left[\frac{1}{2\tilde{\beta}_t}\left\|\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) - \boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t)\right\|^2\right]$$

Target Mean

$$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{1}{\sqrt{\alpha_t}} \cdot \mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{\alpha_t(1-\bar{\alpha}_t)}} \cdot \boldsymbol{\epsilon}$$

Model Mean

$$\boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t) = \frac{1}{\sqrt{\alpha_t}} \cdot \mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{\alpha_t(1-\bar{\alpha}_t)}} \cdot \boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t)$$

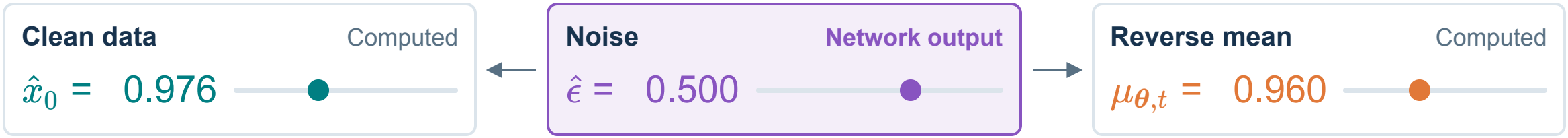
Noise-Prediction Loss

$$\begin{aligned}\mathcal{L}_t &= \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)}\left[\frac{(1-\alpha_t)^2}{2\tilde{\beta}_t\alpha_t(1-\bar{\alpha}_t)}\left\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t)\right\|^2\right] \\ &= \mathbb{E}_{\boldsymbol{\epsilon}\sim\mathcal{N}(0,\mathbf{I})}\left[\frac{(1-\alpha_t)^2}{2\tilde{\beta}_t\alpha_t(1-\bar{\alpha}_t)}\left\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta},t}\left(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}\right)\right\|^2\right]\end{aligned}$$

> Train the network to predict the Gaussian noise $\boldsymbol{\epsilon}$ used to construct $\mathbf{x}_t$.

Prediction Targets and Loss Functions

Linked predictions, one coordinate: $x_t = 1, \quad \alpha_t = 0.6, \quad \bar{\alpha}_t = 0.39$



Network output at $\mathbf{x}_t$		Training target	Squared error
Clean data	$\hat{\mathbf{x}}_{\boldsymbol{\theta},t}$	$\mathbf{x}_0$	$\ell_{x_0} = \lVert\hat{\mathbf{x}}_{\boldsymbol{\theta},t} - \mathbf{x}_0\rVert^2$
Noise	$\boldsymbol{\epsilon}_{\boldsymbol{\theta},t}$	$\boldsymbol{\epsilon}$	$\ell_\epsilon = \lVert\boldsymbol{\epsilon}_{\boldsymbol{\theta},t} - \boldsymbol{\epsilon}\rVert^2$
Reverse mean	$\boldsymbol{\mu}_{\boldsymbol{\theta},t}$	$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0)$	$\ell_\mu = \lVert\boldsymbol{\mu}_{\boldsymbol{\theta},t} - \tilde{\boldsymbol{\mu}}_t\rVert^2$

For linked predictions, the same noise MSE, with $q = q(\mathbf{x}_t|\mathbf{x}_0)$:

$$\mathcal{L}_t^{(\epsilon)} = \mathbb{E}_q[\ell_\epsilon] = \mathbb{E}_q\left[\frac{\bar{\alpha}_t}{1-\bar{\alpha}_t}\ell_{x_0}\right] = \mathbb{E}_q\left[\frac{\alpha_t(1-\bar{\alpha}_t)}{(1-\alpha_t)^2}\ell_\mu\right]$$

Equivalent with these weights. Plain MSE in each parameterization weights timesteps differently.

Ho J. et al. Denoising Diffusion Probabilistic Models, 2020
Salimans T., Ho J. Progressive Distillation for Fast Sampling of Diffusion Models, 2022

Reparametrization of DDPM

$$\mathcal{L}_{\phi,\theta}(\mathbf{x}) = \mathbb{E}_{q(\mathbf{x}_1|\mathbf{x}_0)} \log p_{\boldsymbol{\theta}}(\mathbf{x}_0|\mathbf{x}_1) - D_{\mathrm{KL}}\big(q(\mathbf{x}_T|\mathbf{x}_0)\|p(\mathbf{x}_T)\big) - \sum_{t=2}^{T} \underbrace{\mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} D_{\mathrm{KL}}\big(q(\mathbf{x}_{t-1}|\mathbf{x}_t,\mathbf{x}_0)\|p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)\big)}_{\mathcal{L}_t}$$

$$\mathcal{L}_t = \mathbb{E}_{\boldsymbol{\epsilon}\sim\mathcal{N}(0,\mathbf{I})}\left[\frac{(1-\alpha_t)^2}{2\tilde{\beta}_t\alpha_t(1-\bar{\alpha}_t)}\left\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta},t}\big(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}\big)\right\|^2\right]$$

- Dropping the time-dependent coefficient **changes the relative weights of noise levels**.
- The reconstruction term ($t = 1$) also uses noise prediction.

Simplified Objective

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t\sim\text{Uniform}\{1,\ldots,T\}}\mathbb{E}_{\boldsymbol{\epsilon}\sim\mathcal{N}(0,\mathbf{I})}\left\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta},t}\big(\sqrt{\bar{\alpha}_t}\cdot\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\cdot\boldsymbol{\epsilon}\big)\right\|^2$$

Outline

01 Diffusion Parameterizations

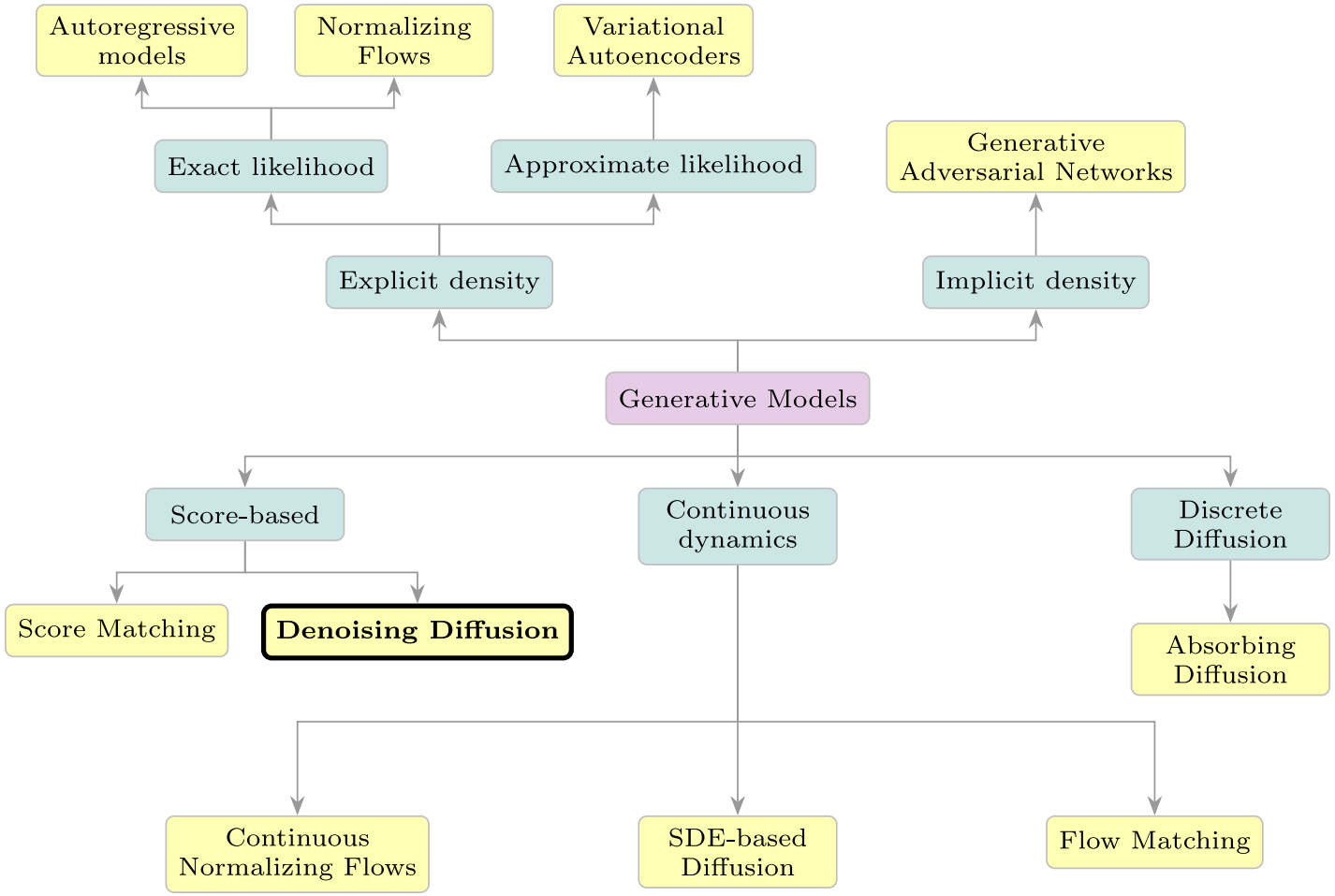
02 Denoising Diffusion Probabilistic Model (DDPM)

03 From DDPM to DDIM

04 Model Guidance

Classifier Guidance
Classifier-Free Guidance

Generative Models Taxonomy



Denoising Diffusion Probabilistic Model (DDPM)

DDPM as a Hierarchical VAE

- **Encoder:** a fixed Gaussian Markov chain $q(\mathbf{x}_1, \ldots, \mathbf{x}_T | \mathbf{x}_0)$.
- **Latents:** $\mathbf{z} = (\mathbf{x}_1, \ldots, \mathbf{x}_T)$; each $\mathbf{x}_t$ has the same dimension as $\mathbf{x}_0$.
- **Decoder:** one Gaussian step $p_{\boldsymbol{\theta}}(\mathbf{x}_0 | \mathbf{x}_1)$.
- **Prior:** a learned Gaussian reverse chain from $p(\mathbf{x}_T) = \mathcal{N}(0, \mathbf{I})$ down to $\mathbf{x}_1$.

Fixed

Sample any $\mathbf{x}_t$ directly from $\mathbf{x}_0$:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\, \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\, \boldsymbol{\epsilon}.$$

Learned

Predict noise to obtain the reverse mean:

$$\boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t) \quad \longrightarrow \quad \boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t).$$

Training: one randomly chosen noise level per example.
Sampling: sequential reverse steps from noise to data.

Denoising Diffusion Probabilistic Model (DDPM)

Training

1. Sample $\mathbf{x}_0 \sim p_{\text{data}}(\mathbf{x})$, $t \sim \text{Uniform}\{1, \dots, T\}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$.
2. Compute noisy image $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \cdot \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \cdot \boldsymbol{\epsilon}$.
3. Minimize $\mathcal{L}_{\text{simple}} = \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t)\|^2$ by SGD.

Sampling (Ancestral)

1. Sample $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$.
2. For $t = T, \dots, 2$, compute the mean and sample the reverse step:

$$\boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t) = \frac{\mathbf{x}_t}{\sqrt{\alpha_t}} - \frac{1 - \alpha_t}{\sqrt{\alpha_t(1 - \bar{\alpha}_t)}} \boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t),$$

$$\mathbf{x}_{t-1} = \boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \sqrt{\tilde{\beta}_t} \boldsymbol{\epsilon}_t, \qquad \boldsymbol{\epsilon}_t \sim \mathcal{N}(0, \mathbf{I}).$$

3. Return the decoder mean $\hat{\mathbf{x}}_0 = \boldsymbol{\mu}_{\boldsymbol{\theta},1}(\mathbf{x}_1)$.

From Denoising Score Matching to DDPM

DSM: learn $\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) \approx \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t)$ using conditional targets:

$$\ell_{\boldsymbol{\theta}}(t) = \mathbb{E}_{p_{\text{data}}(\mathbf{x}_0)} \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} \left\| \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) - \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t|\mathbf{x}_0) \right\|_2^2 .$$

Gaussian perturbation: $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$.

$$\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t|\mathbf{x}_0) = -\frac{\mathbf{x}_t - \sqrt{\bar{\alpha}_t}\mathbf{x}_0}{1-\bar{\alpha}_t} = -\frac{\boldsymbol{\epsilon}}{\sqrt{1-\bar{\alpha}_t}}.$$

Parameterize the score through noise prediction and substitute into DSM:

$$\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) = -\frac{\boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t)}{\sqrt{1-\bar{\alpha}_t}} \quad \Longrightarrow \quad \ell_{\boldsymbol{\theta}}(t) = \frac{\mathbb{E}_{\mathbf{x}_0,\boldsymbol{\epsilon}} \left\| \boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t) - \boldsymbol{\epsilon} \right\|_2^2}{1-\bar{\alpha}_t}.$$

We recover the DDPM simple loss, with $t \sim \text{Uniform}\{1, \dots, T\}$:

$$\mathbb{E}_t[(1-\bar{\alpha}_t)\,\ell_{\boldsymbol{\theta}}(t)] = \mathbb{E}_{t,\mathbf{x}_0,\boldsymbol{\epsilon}} \left\| \boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t) - \boldsymbol{\epsilon} \right\|_2^2 = \mathbb{E}[\mathcal{L}_{\text{simple}}].$$

Vincent P. A Connection Between Score Matching and Denoising Autoencoders, 2010
Ho J., et al. Denoising Diffusion Probabilistic Models, 2020

DDPM vs NCSN: Objectives

For a chosen perturbation $q(\mathbf{x}_t|\mathbf{x}_0)$, define the conditional score error:

$$\ell_{\boldsymbol{\theta}}(t) = \mathbb{E}_{p_{\text{data}}(\mathbf{x}_0)}\mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} \left\| \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) - \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t|\mathbf{x}_0) \right\|^2 .$$

Same DSM form: compare weights and perturbations

$$\mathcal{L} = \mathbb{E}_{t\sim\text{Uniform}\{1,\ldots,T\}} \left[w_t \, \ell_{\boldsymbol{\theta}}(t) \right] .$$

Objective	Gaussian perturbation	Conditional score target	Weight w_t
DDPM simple loss	$\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}$	$-\boldsymbol{\epsilon}/\sqrt{1-\bar{\alpha}_t}$	$1-\bar{\alpha}_t$
NCSN	$\mathbf{x}_0 + \sigma_t\boldsymbol{\epsilon}$	$-\boldsymbol{\epsilon}/\sigma_t$	σ_t^2

Changing the perturbation changes the inputs, targets and $\ell_\theta(t)$ itself.

For a fixed perturbation, replacing the DDPM ELBO weights also changes the objective.
First connect weights to time sampling; then connect the perturbations.

Ho J., et al. Denoising Diffusion Probabilistic Models, 2020
Song Y., Ermon S. Generative Modeling by Estimating Gradients of the Data Distribution, 2019

Loss Weighting and Time Sampling

Fix the perturbation and its per-step loss $\ell_{\boldsymbol{\theta}}(t)$; let $w_t > 0$.

$$\mathcal{L} = \frac{1}{T} \sum_{t=1}^{T} w_t \ell_{\boldsymbol{\theta}}(t).$$

Sampling a noise level more often gives it more weight

$$\mathbb{E}_{t\sim r}[\ell_{\boldsymbol{\theta}}(t)] = \sum_{t=1}^{T} r_t \ell_{\boldsymbol{\theta}}(t) = \mathbb{E}_{t\sim \mathrm{Uniform}\{1,\dots,T\}}[T r_t \, \ell_{\boldsymbol{\theta}}(t)] \,.$$

Conversely, choose $r_t = w_t / \sum_s w_s$ to absorb the weights, up to an overall loss scale.

To change time sampling while preserving a chosen objective, compensate:

$$\mathcal{L} = \mathbb{E}_{t\sim r}\left[\frac{w_t}{T r_t} \ell_{\boldsymbol{\theta}}(t)\right], \qquad r_t > 0.$$

The objective stays the same; gradient variance can change. These are **training** times.

DDPM and NCSN: Connecting Noise Coordinates

DDPM scales the clean signal as it adds noise:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I}).$$

Divide by the signal coefficient

$$\underbrace{\frac{\mathbf{x}_t}{\sqrt{\bar{\alpha}_t}}}_{\tilde{\mathbf{x}}_t} = \mathbf{x}_0 + \underbrace{\sqrt{\frac{1-\bar{\alpha}_t}{\bar{\alpha}_t}}}_{\sigma_t}\,\boldsymbol{\epsilon}.$$

Rescale the score and its loss

Both the conditional score target and the model scale by $\sqrt{\bar{\alpha}_t}$:

$$\tilde{\mathbf{s}}_{\boldsymbol{\theta},t}(\tilde{\mathbf{x}}_t) = \sqrt{\bar{\alpha}_t}\,\mathbf{s}_{\boldsymbol{\theta},t}(\sqrt{\bar{\alpha}_t}\,\tilde{\mathbf{x}}_t), \qquad \tilde{\ell}_{\boldsymbol{\theta}}(t) = \bar{\alpha}_t\,\ell_{\boldsymbol{\theta}}(t).$$

Here $\tilde{\ell}_{\boldsymbol{\theta}}(t)$ is the DSM error in the rescaled coordinates. Hence

$$(1-\bar{\alpha}_t)\,\ell_{\boldsymbol{\theta}}(t) = \sigma_t^2\,\tilde{\ell}_{\boldsymbol{\theta}}(t).$$

The weighted losses agree at each matched noise level.
Next, compare the sampling rules: ancestral updates and Langevin dynamics.

Song J., Meng C., Ermon S. Denoising Diffusion Implicit Models, 2021
Song Y. et al. Score-Based Generative Modeling through Stochastic Differential Equations, 2020

DDPM: Reverse Mean in Score Form

To compare DDPM and Langevin updates, express the DDPM mean in terms of the score.

$$\boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t) = \frac{\mathbf{x}_t}{\sqrt{\alpha_t}} - \frac{1-\alpha_t}{\sqrt{\alpha_t(1-\bar{\alpha}_t)}}\boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t).$$

$$\boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t) = \frac{\mathbf{x}_t}{\sqrt{\alpha_t}} + \frac{1-\alpha_t}{\sqrt{\alpha_t}} \underbrace{\left(-\frac{\boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t)}{\sqrt{1-\bar{\alpha}_t}}\right)}_{\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t)}.$$

$$\boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t) = \frac{\mathbf{x}_t + \beta_t\, \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t)}{\sqrt{\alpha_t}}, \qquad \beta_t = 1-\alpha_t.$$

Sampling (Ancestral)

$$\mathbf{x}_{t-1} = \boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \sqrt{\tilde{\beta}_t}\,\boldsymbol{\epsilon}_t, \qquad \boldsymbol{\epsilon}_t \sim \mathcal{N}(0,\mathbf{I}), \quad t \geq 2.$$

Ho J., et al. Denoising Diffusion Probabilistic Models, 2020
Song Y., et al. Score-Based Generative Modeling through Stochastic Differential Equations, 2021

DDPM vs NCSN: Sampling

Sampling (Ancestral)

1. Compute **one DDPM transition** from t to $t-1$ ($t \geq 2$).

$$\mathbf{x}_{t-1} = \boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \sqrt{\tilde{\beta}_t}\,\boldsymbol{\epsilon}_t = \frac{\mathbf{x}_t + \beta_t\,\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t)}{\sqrt{\alpha_t}} + \sqrt{\tilde{\beta}_t}\,\boldsymbol{\epsilon}_t.$$

Sampling (Annealed Langevin)

1. Run L **NCSN updates at fixed** σ_t, $l = 1, \ldots, L$.

$$\mathbf{x}_t^l = \mathbf{x}_t^{l-1} + \frac{\eta_t}{2}\,\mathbf{s}_{\boldsymbol{\theta},\sigma_t}(\mathbf{x}_t^{l-1}) + \sqrt{\eta_t}\,\boldsymbol{\epsilon}_t^l.$$

2. Set $\mathbf{x}_{t-1}^0 := \mathbf{x}_t^L$ and lower the noise level ($t > 1$).

All step noises are independent $\mathcal{N}(0, \mathbf{I})$ samples.

> **DDPM:** ancestral transitions between noise levels.
> **NCSN:** Langevin MCMC at each fixed noise level.

Ho J., et al. Denoising Diffusion Probabilistic Models, 2020
Song Y., et al. Score-Based Generative Modeling through Stochastic Differential Equations, 2021

DDPM vs NCSN: Summary

Summary

- Gaussian perturbations:
  - DDPM: $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \cdot \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \cdot \boldsymbol{\epsilon}$;
  - NCSN: $\mathbf{x}_t = \mathbf{x}_0 + \sigma_t \cdot \boldsymbol{\epsilon}$.
  - Use $\tilde{\mathbf{x}}_t = \mathbf{x}_t / \sqrt{\bar{\alpha}_t}$ and $\sigma_t^2 = (1 - \bar{\alpha}_t)/\bar{\alpha}_t$ to recover NCSN.
- Scale target and model scores by $\sqrt{\bar{\alpha}_t}$ to obtain the rescaled DSM error:

$$\tilde{\ell}_{\boldsymbol{\theta}}(t) = \bar{\alpha}_t \ell_{\boldsymbol{\theta}}(t), \qquad \sigma_t^2 \tilde{\ell}_{\boldsymbol{\theta}}(t) = (1 - \bar{\alpha}_t) \ell_{\boldsymbol{\theta}}(t).$$

- Equivalent training objectives do not fix the sampling rule:
  - DDPM: one reverse transition per noise level;
  - NCSN: L Langevin updates at each fixed noise level;
  - A trained score model can support different samplers.

Kingma D. et al. Variational Diffusion Models, 2021
Song Y. et al. Score-Based Generative Modeling through Stochastic Differential Equations, 2020

Outline

01 Diffusion Parameterizations

02 Denoising Diffusion Probabilistic Model (DDPM)

03 From DDPM to DDIM

04 Model Guidance

Classifier Guidance
Classifier-Free Guidance

DDIM: One Noise, a Whole Trajectory

The simple loss uses **one noise level at a time**, not a complete path:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{\mathbf{x}_0,t,\boldsymbol{\epsilon}}\left[\left\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t)\right\|^2\right].$$

Choose one noise vector for the whole path

Draw $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$ once and use it at **every** noise level:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}, \qquad t = 1, \ldots, T.$$

Same Gaussian marginals $q(\mathbf{x}_t \mid \mathbf{x}_0)$; a different joint $q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)$.

DDPM: the next state uses only $\mathbf{x}_t$ and fresh noise; its origin no longer matters.

> **Why non-Markovian?** Different clean-image/noise pairs can give the same $\mathbf{x}_t$ but different next states. Knowing $\mathbf{x}_0$ identifies the pair and fixes the continuation.
>
> $$q(\mathbf{x}_{t+1} \mid \mathbf{x}_t, \mathbf{x}_0) \neq q(\mathbf{x}_{t+1} \mid \mathbf{x}_t), \qquad 1 \leq t < T.$$

DDIM: Moving Along the Shared-Noise Path

Two noise levels $s < t$, the **same clean image and noise**:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon},$$
$$\mathbf{x}_s = \sqrt{\bar{\alpha}_s}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_s}\,\boldsymbol{\epsilon}.$$

If $\mathbf{x}_0$ were known: eliminate $\boldsymbol{\epsilon}$ using the current $\mathbf{x}_t$.

$$\mathbf{x}_s = \sqrt{\bar{\alpha}_s}\,\mathbf{x}_0 + \sqrt{\frac{1-\bar{\alpha}_s}{1-\bar{\alpha}_t}}\left(\mathbf{x}_t - \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0\right).$$

Given $\mathbf{x}_t$ and $\mathbf{x}_0$, $\mathbf{x}_s$ is fixed: **no new noise is needed**.

During generation: estimate the unknown pair with the trained DDPM predictor.

$$\hat{\boldsymbol{\epsilon}} = \boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t), \qquad \hat{\mathbf{x}}_0 = \frac{\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\hat{\boldsymbol{\epsilon}}}{\sqrt{\bar{\alpha}_t}}.$$

Substitute the estimates into the shared-noise path:

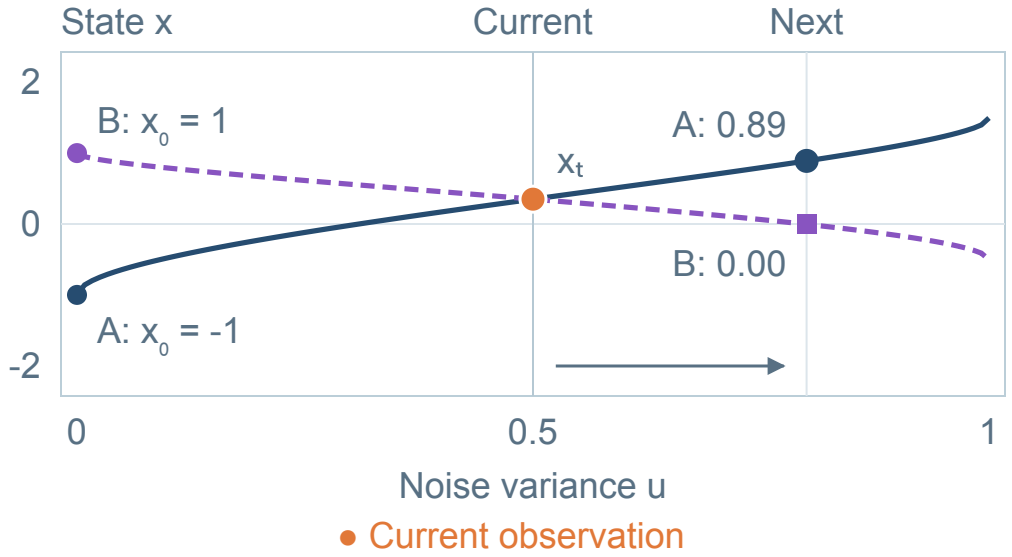
$$\boxed{\mathbf{x}_s = \sqrt{\bar{\alpha}_s}\,\hat{\mathbf{x}}_0 + \sqrt{1-\bar{\alpha}_s}\,\hat{\boldsymbol{\epsilon}}.}$$

The noise schedule is unchanged; the learned sampling step is approximate.

DDIM: Shared Paths and Deterministic Steps

Same training marginals; different transitions. $u = 1 - \bar{\alpha}$

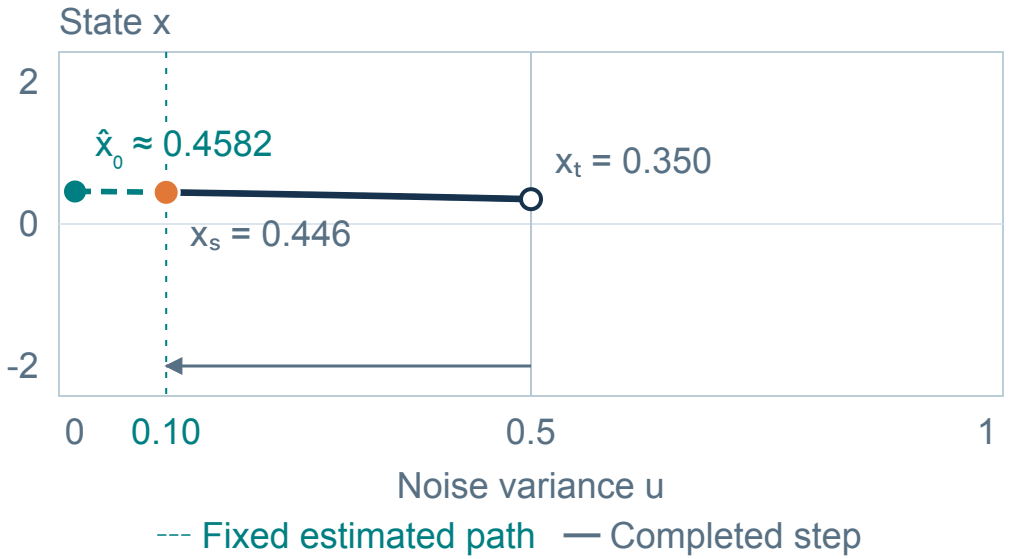
Forward: reveal the clean origin



Only x_t: A 27.1%, B 72.9%.

Knowing x_0 selects one continuation.

DDIM: one step with a fixed pair



Keep $(\hat{x}_0, \hat{\epsilon}) \approx (0.4582, 0.037)$ fixed.

$x_s \approx \sqrt{0.90} \cdot 0.4582 + \sqrt{0.10} \cdot 0.037 \approx 0.446$

> Predict a pair → fix the path → choose lower noise → move without fresh noise → predict again.

Equal-probability data $x_0 = \pm 1$. Analytic MSE denoiser; no training. Start at an intermediate noise level.

DDIM: Deterministic Sampling

Sampling (DDIM)

1. Sample $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$; choose a grid $T = t_K > \cdots > t_0 = 0$.
2. For $k = K, \ldots, 1$, set $t = t_k$, $s = t_{k-1}$ and compute:

$$\hat{\boldsymbol{\epsilon}} = \boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t) \qquad \text{predict noise,}$$

$$\hat{\mathbf{x}}_0 = \frac{\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\hat{\boldsymbol{\epsilon}}}{\sqrt{\bar{\alpha}_t}} \qquad \text{estimate clean data,}$$

$$\mathbf{x}_s = \sqrt{\bar{\alpha}_s}\,\hat{\mathbf{x}}_0 + \sqrt{1-\bar{\alpha}_s}\,\hat{\boldsymbol{\epsilon}} \qquad \text{update state.}$$

3. Return $\mathbf{x}_0$.

Deterministic steps: randomness comes from $\mathbf{x}_T$; recompute both estimates at every step.

Markov sampler: the next state depends only on the current $\mathbf{x}_t$ and the chosen times.

Fewer steps: choose $K \ll T$ on the original schedule: K network evaluations, no retraining.

Outline

01 Diffusion Parameterizations

02 Denoising Diffusion Probabilistic Model (DDPM)

03 From DDPM to DDIM

04 Model Guidance

Classifier Guidance
Classifier-Free Guidance

Guidance

- **Unconditional generation:** $p_{\theta}(\mathbf{x})$. **Conditional generation:** $p_{\theta}(\mathbf{x}|\mathbf{y})$.
- The condition $\mathbf{y}$ can be a class label, a text prompt, or reference images.
- **Guidance** modifies sampling to strengthen the influence of $\mathbf{y}$.



FLUX.2 · Text prompt (abridged)
Black cat behind a watermelon.



FLUX.2 · Text prompt (abridged)
Sushi restaurant with a red "OPEN" sign.

Ho J., Salimans T. Classifier-Free Diffusion Guidance, 2022.
Black Forest Labs. FLUX.2 Prompting Guide. Published model outputs; text conditions abridged.

Conditional Generation

Given a condition $\mathbf{y}$, generate $\mathbf{x} \sim p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{y})$ with model parameters $\boldsymbol{\theta}$.

Condition $\mathbf{y}$	Generated $\mathbf{x}$	Tasks
Class label or text prompt	**Image**	Class-conditional / text-to-image
Image (+ mask / instruction)	**Image**	Editing, inpainting, super-resolution
Text (+ reference image)	**Video**	Text-to-video / image-to-video
Prompt + context	Text / code	Chat, translation, summarization, coding
Image (+ question)	Text	Captioning / visual question answering
Speech	Text	Speech recognition
Text	Audio	Speech synthesis, music, sound effects

Ouyang L. et al. InstructGPT, 2022. · Chen M. et al. Codex, 2021. · Li J. et al. BLIP-2, 2023. · Liu H. et al. AudioLDM, 2023.
Brooks T. et al. InstructPix2Pix, 2023. · Rombach R. et al. Latent Diffusion, 2022. · Wan Team. Wan, 2025.

Label Guidance

Label: Ostrich (10th ImageNet class)



VQ-VAE (Proposed) BigGAN deep

Razavi A., Oord A., et al. Generating Diverse High-Fidelity Images with VQ-VAE-2, 2019

Text Guidance

Prompt: Anna
SDXL fine-tuned on Disney princesses. Same initial noise; only γ changes.

$\gamma = 3.5$ $\gamma = 4.5$ $\gamma = 5.5$ $\gamma = 6.5$



Kasymov A., et al. AutoLoRA: AutoGuidance Meets Low-Rank Adaptation for Diffusion Models, 2024. Fig. 3, top row (selected columns). CC BY 4.0.

Guidance in Generative Models

How to make a conditional model?

Instead of sampling from $p_{\boldsymbol{\theta}}(\mathbf{x})$, we sample from $p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{y})$.

Given **supervised** data $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^n$, we can treat $\mathbf{y}$ as an additional model input:

- $p_{\boldsymbol{\theta}}(x_j|\mathbf{x}_{1:j-1}, \mathbf{y})$ for AR models;
- Encoder $q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}, \mathbf{y})$ and decoder $p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z}, \mathbf{y})$ for VAEs;
- $G_{\boldsymbol{\theta}}(\mathbf{z}, \mathbf{y})$ for NFs and GANs;
- $p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{y})$ for DDPMs.

Challenge

- Empirically, images sampled with this procedure do not fit well enough to the desired label $\mathbf{y}$.
- Being able to control the strength of guidance is especially valuable.

Outline

01 Diffusion Parameterizations

02 Denoising Diffusion Probabilistic Model (DDPM)

03 From DDPM to DDIM

04 Model Guidance

Classifier Guidance

Classifier-Free Guidance

Classifier Guidance

Sampling (Ancestral)

1. Sample fresh $\boldsymbol{\epsilon}_t \sim \mathcal{N}(0, \mathbf{I})$ and update ($t \geq 2$).

$$\mathbf{x}_{t-1} = \frac{\mathbf{x}_t + \beta_t \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t)}{\sqrt{\alpha_t}} + \sqrt{\tilde{\beta}_t}\, \boldsymbol{\epsilon}_t.$$

The noise predictor defines an **estimate** of the noisy-data score:

$$\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) = -\frac{\boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t)}{\sqrt{1-\bar{\alpha}_t}} \approx \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t).$$

Conditional Generation

Use $\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y}) \approx \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t|\mathbf{y})$ in the same step:

$$\mathbf{x}_{t-1} = \frac{\mathbf{x}_t + \beta_t \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y})}{\sqrt{\alpha_t}} + \sqrt{\tilde{\beta}_t}\, \boldsymbol{\epsilon}_t.$$

How can we relate $q(\mathbf{x}_t)$ and $q(\mathbf{x}_t|\mathbf{y})$?

Classifier Guidance: Guided Score Function

For now, restrict $\mathbf{y}$ to **class labels**. By Bayes' theorem,

$$q(\mathbf{x}_t|\mathbf{y}) = \frac{q(\mathbf{x}_t)q(\mathbf{y}|\mathbf{x}_t)}{q(\mathbf{y})}.$$

$$\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t|\mathbf{y}) = \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log q(\mathbf{y}|\mathbf{x}_t).$$

Classifier-Guided Estimate

Train a noisy-data classifier $p(\mathbf{y}|\mathbf{x}_t) \approx q(\mathbf{y}|\mathbf{x}_t)$ at every noise level, providing t as an input.

$$\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y}) := \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\mathbf{x}_t) \approx \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t|\mathbf{y}).$$

Guidance Scale

$$\mathbf{s}^{\gamma}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y}) := \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \gamma \cdot \nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\mathbf{x}_t).$$

γ controls guidance strength: $\gamma = 0$ gives the unconditional estimate; $\gamma = 1$ gives the conditional estimate.

Classifier Guidance: Distribution Sharpening

Scaled Guided Score Estimate

$$\mathbf{s}^{\gamma}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y}) = \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \gamma \nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\mathbf{x}_t).$$

Exact Scores at a Fixed Noise Level

$$\begin{aligned}\mathbf{s}^{\gamma}_{t}(\mathbf{x}_t, \mathbf{y}) &:= \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t) + \gamma \nabla_{\mathbf{x}_t} \log q(\mathbf{y}|\mathbf{x}_t) \\ &= \nabla_{\mathbf{x}_t} \log q_{\gamma}(\mathbf{x}_t|\mathbf{y}),\end{aligned}$$

$$q_{\gamma}(\mathbf{x}_t|\mathbf{y}) := \frac{q(\mathbf{x}_t)q(\mathbf{y}|\mathbf{x}_t)^{\gamma}}{Z_t(\mathbf{y}, \gamma)}.$$

Larger γ favors samples with higher class probability; very large values can reduce diversity and introduce artifacts.

This fixed-level identity does not specify the final distribution of the guided sampler.

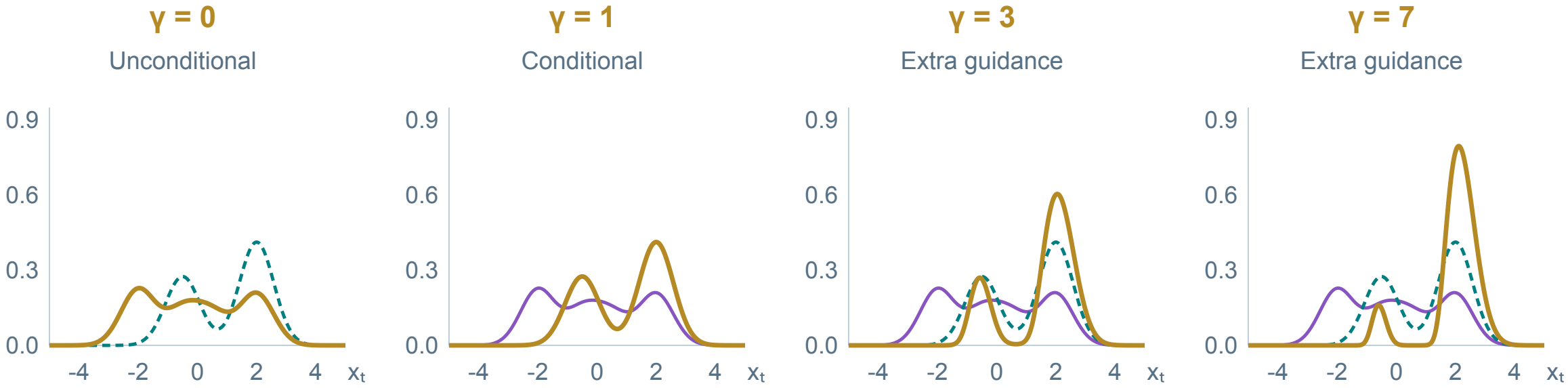
Dhariwal P., Nichol A. Diffusion Models Beat GANs on Image Synthesis, 2021
Sadat S. et al. Eliminating Oversaturation and Artifacts of High Guidance Scales in Diffusion Models, 2025

Guidance Scale Changes the Density

$$q_\gamma(x_t|y) \propto q(x_t)\, q(y|x_t)^\gamma \qquad \text{at a fixed noise level.}$$

Same one-dimensional toy and target class B; only γ changes.

$q(x_t)$ $q(x_t|y)$ · dashed $q_\gamma(x_t|y)$



Increasing γ shifts mass toward regions with higher classifier confidence.

A 1D Gaussian-mixture example at a fixed noise level. This plot describes a density at that level.

Classifier Guidance: Overview

Training

1. Train the DDPM as before.
2. Train an additional classifier $p(\mathbf{y}|\mathbf{x}_t)$ on noisy data (time-dependent).

Sampling (Guided)

1. Sample $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$.
2. For $t = T, \dots, 2$, compute the guided mean and sample the reverse step:

$$\boldsymbol{\mu}^{\gamma}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y}) = \frac{\mathbf{x}_t + \beta_t \, \mathbf{s}^{\gamma}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y})}{\sqrt{\alpha_t}},$$

$$\mathbf{x}_{t-1} = \boldsymbol{\mu}^{\gamma}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y}) + \sqrt{\tilde{\beta}_t}\, \boldsymbol{\epsilon}_t, \qquad \boldsymbol{\epsilon}_t \sim \mathcal{N}(0, \mathbf{I}).$$

3. Return the guided decoder mean $\hat{\mathbf{x}}_0 = \boldsymbol{\mu}^{\gamma}_{\boldsymbol{\theta},1}(\mathbf{x}_1, \mathbf{y})$.

Outline

01 Diffusion Parameterizations

02 Denoising Diffusion Probabilistic Model (DDPM)

03 From DDPM to DDIM

04 Model Guidance

Classifier Guidance

Classifier-Free Guidance

Classifier-Free Guidance

We want guidance without training an additional noisy-data classifier $p(\mathbf{y}|\mathbf{x}_t)$.

Bayes Theorem

$$\begin{aligned}\nabla_{\mathbf{x}_t} \log q(\mathbf{y}|\mathbf{x}_t) &= \nabla_{\mathbf{x}_t} \log \left(\frac{q(\mathbf{x}_t|\mathbf{y})q(\mathbf{y})}{q(\mathbf{x}_t)} \right) \\ &= \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t|\mathbf{y}) - \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t)\end{aligned}$$

Scaled Guided Score Function

$$\begin{aligned}\mathbf{s}_t^{\gamma}(\mathbf{x}_t, \mathbf{y}) &= \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t) + \gamma \cdot \nabla_{\mathbf{x}_t} \log q(\mathbf{y}|\mathbf{x}_t) \\ &= \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t) + \gamma \cdot \left(\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t|\mathbf{y}) - \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t) \right) \\ &= (1-\gamma) \cdot \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t) + \gamma \cdot \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t|\mathbf{y})\end{aligned}$$

> The gradient of the class log-probability is the difference between conditional and unconditional scores.
> **Learn both scores and combine their estimates; no separate classifier is needed.**

Classifier-Free Guidance: Formulation

Scaled Guided Score Function

$$\mathbf{s}_t^{\gamma}(\mathbf{x}_t, \mathbf{y}) = (1-\gamma) \cdot \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t) + \gamma \cdot \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t|\mathbf{y})$$

Replace the exact scores by their learned estimates:

$$\mathbf{s}_{\boldsymbol{\theta},t}^{\gamma}(\mathbf{x}_t, \mathbf{y}) = (1-\gamma) \cdot \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \gamma \cdot \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y})$$

Naive training approach

- Train an unconditional score model $\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t)$.
- Train a conditional score model $\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y})$.

Sampling (Guided)

Compute a guided DDPM step ($t \geq 2$).

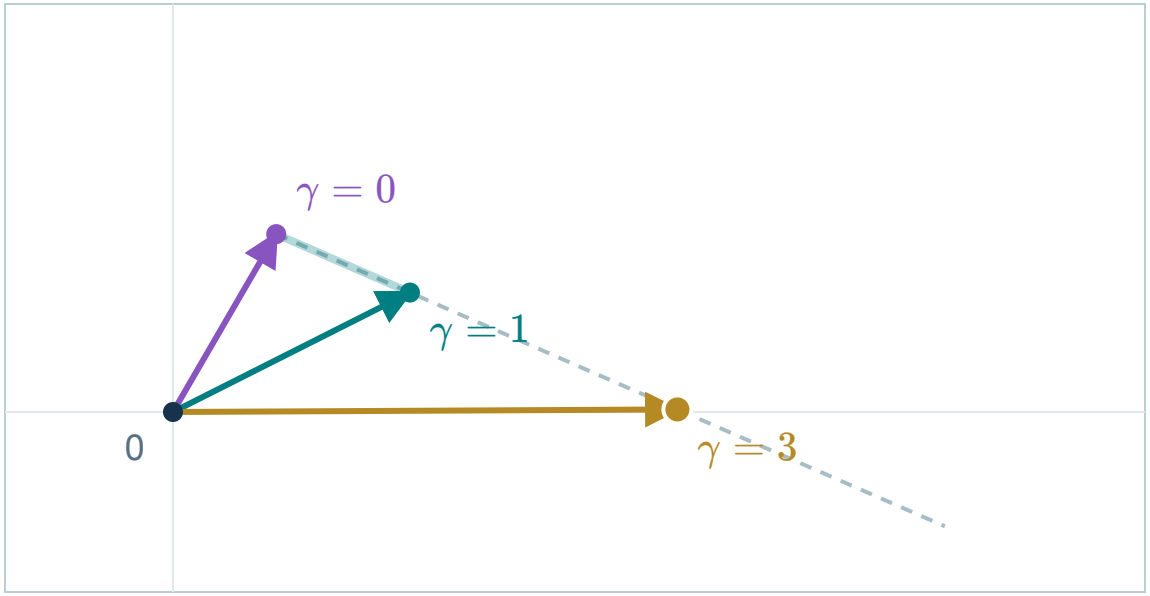
$$\mathbf{x}_{t-1} = \frac{\mathbf{x}_t + \beta_t \mathbf{s}_{\boldsymbol{\theta},t}^{\gamma}(\mathbf{x}_t, \mathbf{y})}{\sqrt{\alpha_t}} + \sqrt{\tilde{\beta}_t}\, \boldsymbol{\epsilon}_t$$

How to avoid training two separate score function models?

CFG: Interpolation and Extrapolation

$$\mathbf{s}_t^{\gamma} = \mathbf{s}_t(\mathbf{x}_t, \varnothing) + \gamma\big(\mathbf{s}_t(\mathbf{x}_t, \mathbf{y}) - \mathbf{s}_t(\mathbf{x}_t, \varnothing)\big)$$

Same two scores: $\gamma = 0$, $\gamma = 1$ and $\gamma = 3$.



Score space · endpoints of vectors

The two base scores are fixed; γ moves the guided endpoint along their line.

$\gamma = 3.0$ · Extrapolation

$\gamma = 0$

$\mathbf{s}_t^0 = \mathbf{s}_t(\mathbf{x}_t, \varnothing)$

$\gamma = 1$

$\mathbf{s}_t^1 = \mathbf{s}_t(\mathbf{x}_t, \mathbf{y})$

$\gamma > 1$

Continue beyond the conditional endpoint.

$\varnothing$ denotes no condition. Analytic toy scores at one fixed observation and noise level.

For $\gamma > 1$, CFG extrapolates beyond the conditional score.

Classifier-Free Guidance

$$\mathbf{s}^{\gamma}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y}) = (1-\gamma)\cdot \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \gamma \cdot \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y})$$

CFG

1. Introduce the "absence of conditioning" label $\mathbf{y} = \varnothing$.
2. Identify the unconditional score model $\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) = \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \varnothing)$.
3. Train one model on paired data, replacing $\mathbf{y}$ with $\varnothing$ with fixed probability.
4. Evaluate the model twice per sampling step: with $\mathbf{y}$ and with $\varnothing$.

Sampling (Guided)

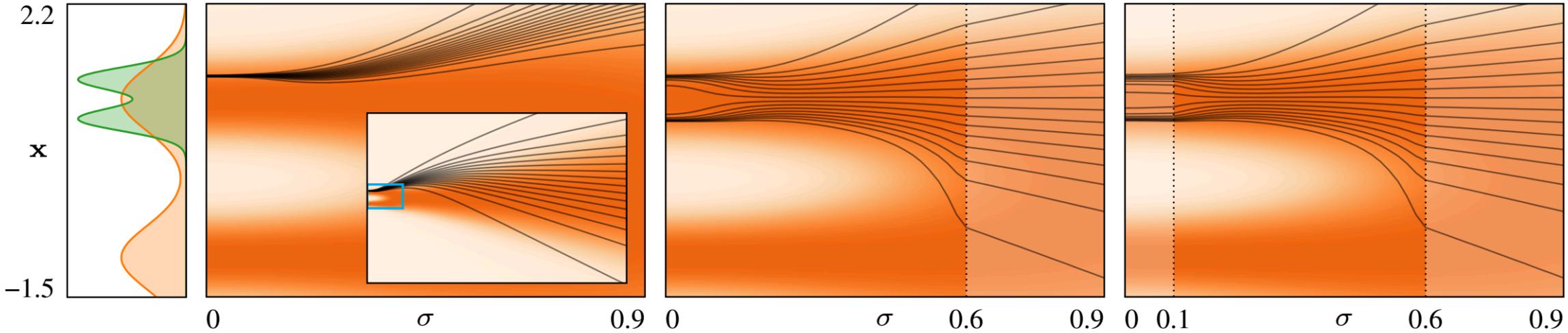
1. Update with fresh $\boldsymbol{\epsilon}_t \sim \mathcal{N}(0, \mathbf{I})$ for $t = T, \ldots, 2$.

$$\mathbf{x}_{t-1} = \frac{\mathbf{x}_t + \beta_t \mathbf{s}^{\gamma}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y})}{\sqrt{\alpha_t}} + \sqrt{\tilde{\beta}_t}\, \boldsymbol{\epsilon}_t.$$

2. Return $\hat{\mathbf{x}}_0 = \boldsymbol{\mu}^{\gamma}_{\boldsymbol{\theta},1}(\mathbf{x}_1, \mathbf{y})$, as before.

Classifier-Free Guidance: Guidance Interval

Apply extra guidance only at intermediate noise levels.



(a) PDFs (b) Guidance everywhere (c) Guidance at $\sigma \in (0, 0.6]$ (d) Guidance at $\sigma \in (0.1, 0.6]$

Toy example ($\gamma = 6$): guidance everywhere drops a mode; limiting the interval restores both.

Outside the interval, $\gamma_t = 1$: **conditioning remains**. Sampling runs **right to left** ($\sigma \downarrow$).

Kynkäänniemi T. et al. Applying Guidance in a Limited Interval Improves Sample and Distribution Quality in Diffusion Models, 2024. Fig. 2.

Classifier-Free Guidance: Guidance Distillation

Learn the guided prediction in a single model evaluation.

Teacher: two evaluations

$$\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \varnothing), \quad \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y})$$

Combine predictions using γ.

distill $\longrightarrow$

Student: one evaluation

$$\mathbf{s}_{\boldsymbol{\phi},t}(\mathbf{x}_t, \mathbf{y}, \gamma)$$

Pass γ as an extra input.

$$\mathbf{s}_{\boldsymbol{\phi},t}(\mathbf{x}_t, \mathbf{y}, \gamma) \approx (1-\gamma)\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \varnothing) + \gamma \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y})$$

Training

1. Sample $(\mathbf{x}_0, \mathbf{y}) \sim p_{\text{data}}$, $t \sim \text{Uniform}\{1, \dots, T\}$, $\gamma \sim \text{Uniform}[\gamma_{\min}, \gamma_{\max}]$.
2. Sample $\mathbf{x}_t \sim q(\mathbf{x}_t|\mathbf{x}_0)$ and compute the **fixed teacher target** $\mathbf{s}^{\gamma}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y})$.
3. Update only $\boldsymbol{\phi}$ to minimize $\mathcal{L} = \|\mathbf{s}_{\boldsymbol{\phi},t}(\mathbf{x}_t, \mathbf{y}, \gamma) - \mathbf{s}^{\gamma}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y})\|_2^2$.

Sampling

1. Evaluate the student once per step; keep the sampler and step count. Used in **FLUX.2 [dev]**.

Meng C. et al. On Distillation of Guided Diffusion Models, 2023.
Black Forest Labs. FLUX.2 [dev] model card: guidance distillation.

Summary

- Reverse-mean, clean-data and noise predictions are convertible; unweighted losses assign different weights to noise levels.
- DDPM replaces ELBO weights with a simple noise-prediction loss and generates through Gaussian reverse steps.
- DDPM simple loss and NCSN agree at matched noise levels with rescaled scores; loss weights can be absorbed into training-time sampling.
- DDIM reuses the DDPM network: same training marginals, deterministic sampling, fewer steps.
- Guidance makes generation controllable through labels or text prompts.
- Classifier guidance uses an auxiliary noisy-data classifier; classifier-free guidance does not.
- Guidance intervals can improve the quality-diversity trade-off; guidance distillation reduces evaluations per step.